**Paradigmas y Lenguajes de Programación**

**Carrera:** Licenciatura en Gestión de Recursos Tecnológicos
Universidad Gastón Dachary

Año 2026

**Nombre y apellido**: Cristian Collins
**Número de matrícula/usuario:** ugd600249
**Carrera:** Licenciatura en Recursos Tecnológicos
**Materia**: Paradigmas y Lenguajes de Programación
**Profesores**: Ing. Agustín Encina, Ing. Dante Sicardi

UNIVERSIDAD Gastón Dachary

**ACTIVIDAD N° 4**

**Reflexión sobre paradigmas y lenguajes en los prototipos de lógica difusa**

**1. Análisis de paradigmas y lenguajes utilizados**
**Prototipo de detección del estado ambiental (Experiencia 1)**
En esta experiencia se emplea un **paradigma de control basado en lógica difusa tipo Mamdani**, el cual pertenece a un enfoque **declarativo basado en reglas**. Este paradigma permite modelar problemas donde las variables no son exactas sino lingüísticas (por ejemplo: "frío", "cálido", "húmedo"), facilitando una representación más cercana al razonamiento humano.
Tal como se describe en el artículo *(Encina et al., s/f)*, el sistema procesa las entradas (temperatura y humedad) mediante fuzzificación, evaluación de reglas y defuzzificación, generando una salida interpretable como "clima bueno", "normal" o "malo". Este flujo evidencia claramente un modelo basado en reglas del tipo **SI–ENTONCES**, característico de este paradigma.
Desde el punto de vista de los lenguajes:

- **MATLAB (Fuzzy Logic Designer)** se utiliza como entorno de diseño, permitiendo trabajar a un alto nivel de abstracción mediante herramientas gráficas. Esto reduce la complejidad matemática y facilita la validación del sistema antes de su implementación.
- **C++ en Arduino** responde a un **paradigma imperativo**, donde el programador define explícitamente cómo se ejecutan las instrucciones en el hardware. Esto es fundamental en sistemas embebidos, donde se requiere control preciso del tiempo y los recursos.

Se observa aquí una arquitectura clara en dos niveles:

- **Nivel declarativo (diseño):** definición del comportamiento del sistema.
- **Nivel imperativo (implementación):** ejecución eficiente en hardware.

Esta separación constituye una fortaleza metodológica, ya que permite validar el modelo antes de llevarlo al entorno físico. La combinación de paradigmas declarativos e imperativos en un mismo proyecto es una práctica habitual en ingeniería de software moderna, dado que cada paradigma resuelve mejor un nivel distinto del problema (Sebesta, 2018).

Este enfoque evidencia cómo los paradigmas no solo definen la forma de programar, sino también la forma de pensar y modelar problemas complejos, permitiendo separar la lógica conceptual del control físico del sistema.

**Mecanismo de avance autoajustable (Experiencia 2)**
En esta segunda experiencia se mantiene el mismo paradigma de control (lógica difusa Mamdani), pero aplicado a un problema distinto: el seguimiento de un objeto en movimiento.
Las variables de entrada (distancia y velocidad) son continuas, y las decisiones deben ser progresivas y suaves, lo cual justifica nuevamente el uso de lógica difusa. Las reglas del tipo:
"SI distancia ES cerca Y velocidad ES lenta ENTONCES acercar"
permiten modelar el comportamiento de manera intuitiva y flexible.
En cuanto a los lenguajes y herramientas:

- **Scilab (sciFLT)** cumple el mismo rol que MATLAB, pero dentro del ecosistema de software libre, lo cual demuestra la independencia del paradigma respecto de la herramienta utilizada.
- **C++ con librería eFLL en Arduino** introduce una diferencia relevante: permite una **implementación orientada a objetos del sistema difuso**, donde los conjuntos, reglas y variables se modelan como objetos. Esto otorga mayor flexibilidad, reutilización y capacidad de modificación directa del sistema sin necesidad de regenerar código automáticamente.

Desde el punto de vista de los paradigmas, nuevamente se observa la combinación de:

- **Paradigma declarativo (reglas difusas)**
- **Paradigma imperativo (control del hardware)**

Esto evidencia cómo distintos paradigmas pueden coexistir y complementarse dentro de un mismo sistema, permitiendo abordar el problema desde diferentes niveles de abstracción.

## 2. Propuesta de mejora

Una mejora relevante para futuros desarrollos sería la migración del modelo Mamdani al modelo Takagi-Sugeno-Kang (TSK) en aquellos casos donde la salida del sistema sea numérica y requiera eficiencia en tiempo real. El modelo Mamdani, utilizado en ambas experiencias, requiere un proceso de defuzzificación (por ejemplo, el método del centroide), el cual implica integración numérica. En el caso analizado, esto se implementa mediante una aproximación discreta (FIS_RESOLUSION = 101), lo que consume recursos computacionales del microcontrolador. Como alternativa, el modelo TSK propone que cada regla tenga como consecuente una función matemática (por ejemplo: $z = a \cdot x + b \cdot y + c$), lo que permite obtener la salida mediante un promedio ponderado sin necesidad de integración. Los beneficios principales son:

- Reducción significativa del costo computacional
- Mayor frecuencia de muestreo de sensores
- Posibilidad de escalar el sistema con más reglas
- Mejor adaptación a sistemas en tiempo real

Esta migración sería especialmente ventajosa en la Experiencia 2, tal como el propio artículo menciona que el método TSK es más eficiente para sistemas de mayores dimensiones, aunque ellos optaron por Mamdani por la simplicidad de las experiencias. Desde un punto de vista práctico:

- En la Experiencia 1, se recomienda mantener Mamdani, ya que la salida es cualitativa y orientada al usuario.
- En la Experiencia 2, TSK sería más adecuado, ya que la salida (movimiento del motor) es intrínsecamente numérica.

Como contrapartida, se pierde parte de la interpretabilidad lingüística, aunque se mantiene la estructura basada en reglas.

Otra mejora interesante sería aprovechar la **simplicidad y rapidez de desarrollo** que se destacó en la Experiencia 1 (gracias al generador automático de código desde MATLAB) y extender ese mismo enfoque modular a la parte imperativa del código. Concretamente, se podría encapsular los componentes de hardware (sensor DHT11, LED RGB, LCD, etc.) en clases simples de C++ reutilizables. De esta forma se mantiene la velocidad de implementación que ofrece el generador FIS, pero se gana mayor organización, reutilización de código entre ambos prototipos y facilidad de mantenimiento, todo ello alineado con las ventajas del paradigma orientado a objetos que estudia Sebesta (2018).

Desde el punto de vista práctico:

- En la Experiencia 1 facilitaría agregar o cambiar sensores/actuadores sin tocar el núcleo del controlador difuso.
- En la Experiencia 2 permitiría crear una jerarquía limpia de actuadores (motores) que se integre fácilmente con el código generado.

De esta manera se combinan lo mejor de ambos mundos: la **rapidez declarativa** del generador de código y la **modularidad** del paradigma orientado a objetos.

Estas mejoras no solo optimizan el rendimiento y la mantenibilidad del sistema, sino que también demuestran la importancia de seleccionar el paradigma adecuado en función del contexto y los requisitos del problema.

## 3. Conclusión

Los prototipos analizados evidencian una correcta selección de paradigmas y lenguajes en función del problema a resolver. La lógica difusa permite abordar sistemas con incertidumbre de manera eficiente y comprensible,

mientras que el uso de lenguajes imperativos como C++ garantiza una implementación eficiente en hardware embebido.
Asimismo, se observa una clara separación entre el nivel de diseño y el nivel de ejecución, lo cual constituye una buena práctica en ingeniería de software.
En este sentido, se puede afirmar que el uso combinado de paradigmas declarativos e imperativos no solo es adecuado, sino necesario para el desarrollo de sistemas complejos, donde se requiere tanto abstracción como control detallado de la ejecución.
**Esta reflexión demuestra que la selección de paradigmas y lenguajes fue adecuada para los objetivos de cada prototipo, permitiendo lograr soluciones funcionales, eficientes y conceptualmente correctas.**

Referencias

Encina, A., Mousquere, C., Bennesch, E., Kucuk, L., & Syniuk, R. (s/f).
*Lógica difusa con Matlab, SciLab y Arduino: Prototipo para la detección del estado ambiental - mecanismo de avance autoajustable.*

Sebesta, R. W. (2018).
*Concepts of programming languages (11th ed.). Pearson.*